PRACTICE 2

SYMMETRIC CRYPTOGRAPHY

Application design, cryptosystem analysis, and a Mini-AES CBC example

Course	Theory of Codes and Cryptography
Implementation language	Python 3

Daniel Nazaruk | Adam Gryniewski | Lucjan Jacek Pucelak | Anas Rakssa Zargaoui

This report follows the Practice 2 specification and reuses the relevant Python project stack established in Practice 1. AES-CBC is used because the exercise explicitly requests CBC; the design adds a separate authentication check to avoid treating unauthenticated ciphertext as trustworthy.

1. Libraries used

The application is designed as a Python 3 service, consistent with the Flask-based project described in Practice 1. Cryptographic operations are delegated to maintained library implementations rather than handwritten cipher code.

Library / module	Main characteristics	Role in this application
cryptography (third-party)	Provides AES, CBC mode, PKCS#7 padding, PBKDF2-HMAC, HKDF, HMAC, and AES key wrapping.	Encrypts/decrypts the file bytes, derives key-encryption material, wraps the random data key, and authenticates the encrypted container.
secrets (Python standard library; reused from Practice 1)	Uses the operating system cryptographic randomness source.	Generates a fresh 32-byte AES key, a 16-byte IV, and a unique 16-byte salt for each encryption operation.
Flask (third-party; reused from Practice 1)	Lightweight framework for HTTP routes and request handling.	Provides the existing web API interface for submitting text/files and returning encrypted/decrypted results. Flask does not perform the cryptography itself.
pathlib (Python standard library)	Cross-platform path and file handling.	Builds input, temporary-output, encrypted-output, and restored-file paths.

Library scope note: liboqs-python was listed in Practice 1 for ML-KEM key establishment and ML-DSA signatures. It is not required for this assignment's standalone AES-CBC processing, so it is not called by this workflow. It can remain part of a larger hybrid design if that feature is explicitly integrated.

2. Flow diagram of the processes

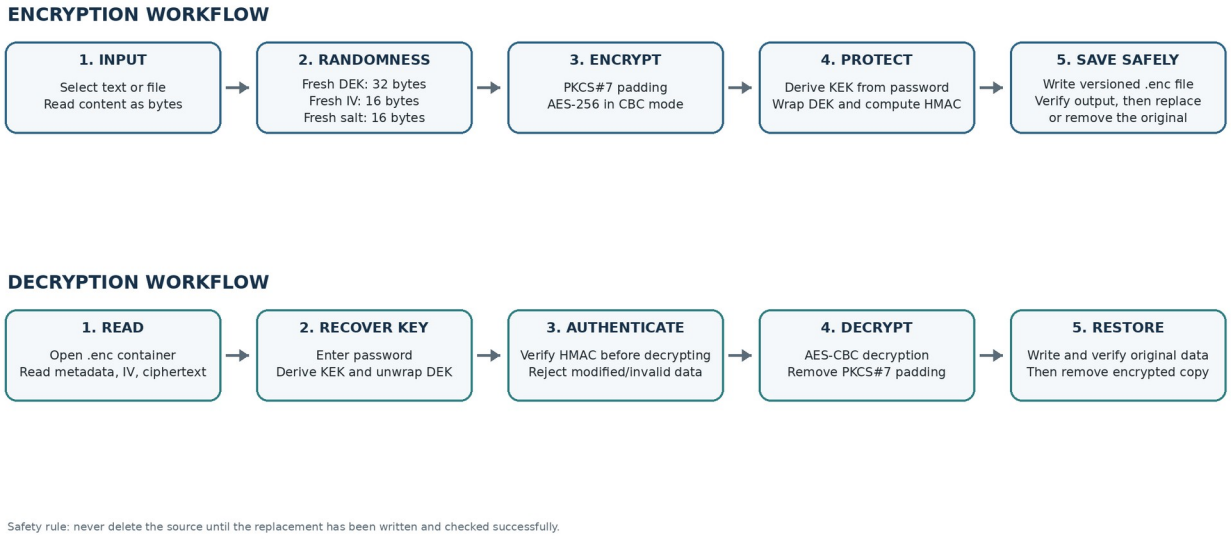


Figure 1. Encryption and decryption workflows, including key protection, ciphertext authentication, and safe replacement of files.

Encryption workflow

1. Accept plaintext or read the selected file in binary mode.
2. Generate a fresh 256-bit data-encryption key (DEK), a fresh 128-bit IV, and a unique salt using secrets.token_bytes().
3. Apply PKCS#7 padding, then encrypt the padded bytes with AES-256-CBC.
4. Derive a key-encryption key (KEK) from the user password, wrap the DEK, and calculate an HMAC-SHA-256 authentication tag over the canonical container fields.
5. Write the complete encrypted container to a new output file and verify that the write succeeded. Only then replace or remove the source file according to the application policy.

Decryption workflow

1. Read the container metadata, salt, wrapped DEK, IV, ciphertext, and authentication tag.
2. Request the password, derive the KEK again, and unwrap the DEK.
3. Recalculate and verify the HMAC tag before decrypting. Reject modified data or an incorrect password; do not overwrite the encrypted input.
4. Decrypt using AES-CBC, remove PKCS#7 padding, and write the recovered data to a new file.
5. Verify the restored output before removing the encrypted copy.

3. Supported file types and encrypted output

The application reads files as bytes, so it does not need to interpret the file format. It can encrypt any file whose bytes can be read, including:

- Text and data files: .txt, .csv, .json
- Images: .jpg, .png, .gif
- Documents and spreadsheets: .pdf, .docx, .xlsx
- Audio and video: .mp3, .mp4, .wav
- Archives and other binary files: .zip, .7z, and executable files

The file extension does not determine whether encryption is possible. For text input, the application converts characters to bytes using UTF-8. For file input, the original bytes are preserved without decoding.

The encrypted output is a binary .enc container with a version/algorithm identifier, KDF identifier and parameters, salt, wrapped DEK, 16-byte IV, ciphertext, and a 32-byte HMAC-SHA-256 tag. The ciphertext length is a multiple of 16 bytes because PKCS#7 padding is applied before AES-CBC. The IV, salt, and KDF parameters are not secret, but they must be stored correctly. The .enc extension is only a naming convention.

File replacement is transactional: write the new output first, flush/close it, and verify success before deleting the input. Ordinary deletion is not a guarantee of secure erasure on SSDs, journaling file systems, snapshots, backups, or cloud-synced folders.

4. Cryptosystem description and justification

Parameter	Selected value	Justification
Algorithm	AES (Advanced Encryption Standard)	A widely deployed symmetric block cipher with standardized, maintained implementations.
Key length	256 bits (32 bytes)	The task requests strong symmetric encryption. The key must be generated securely and protected for later recovery.
Block length	128 bits (16 bytes)	AES has a fixed 128-bit block size for AES-128, AES-192, and AES-256.
Mode	CBC (Cipher Block Chaining)	Required by the assignment and links each plaintext block to the previous ciphertext block. The first block uses the IV.
IV	128 bits; fresh and unpredictable per encryption	Matches the AES block size. The IV need not be secret but must be stored with the ciphertext and not reused with the same key.
Padding	PKCS#7 with a 128-bit block size	Expands the data to a whole number of AES blocks and allows the original length to be recovered after decryption. A full padding block is added when the input already aligns to a block boundary.
Integrity protection	HMAC-SHA-256, verified before decryption	CBC by itself does not detect tampering. The authentication tag protects the container fields and ciphertext against undetected modification.

Parameter	Selected value	Justification
Input/output	Bytes; text encoded as UTF-8	Allows the same data path to support arbitrary files and text input.

CBC is retained to satisfy the exercise, not because it is the preferred mode for a new product. Authenticated encryption such as AES-GCM is usually simpler for new designs because confidentiality and integrity are provided together. With CBC, the authentication tag must be checked before any plaintext is released.

AES is defined by NIST as a block cipher with 128-bit blocks and 128-, 192-, or 256-bit keys. Mini-AES, used in the educational example below, is a separate toy cipher and must never be used to protect real files.

5. Key selection, storage, and security

A new random DEK is generated for each encryption operation with secrets.token_bytes(32). The application also generates a new 16-byte IV and a unique 16-byte salt. The DEK is required to decrypt the file later, so the application must store a protected copy rather than discard it.

The proposed key-protection design uses envelope encryption:

1. Generate a fresh 256-bit DEK and encrypt the padded input with AES-256-CBC.
2. Ask the user for a master password. Derive a 256-bit KEK using PBKDF2-HMAC-SHA-256 with the stored unique salt and a documented, suitably high iteration count.
3. Wrap the DEK with the KEK using the AES key-wrap function from the cryptography package. Store the wrapped key, salt, KDF identifier, and KDF parameters; never store the raw DEK or password in the container.
4. Derive a separate MAC key from the DEK using HKDF-SHA-256 with an application-specific context label. Use it to authenticate the container version, KDF metadata, wrapped key, IV, and ciphertext with HMAC-SHA-256.
5. During decryption, derive the KEK from the supplied password, unwrap the DEK, derive the MAC key, and verify the tag before attempting CBC decryption. Stop on any failure.

The password itself must not be stored or logged. The salt and KDF parameters are public metadata and must be retained so the same KEK can be derived later. Losing the password may make the encrypted data unrecoverable. Normal file deletion does not guarantee secure erasure, so source removal must happen only after the encrypted or restored output has been written and validated.

Connection to Practice 1: the symmetric DEK continues to protect file contents. A future integration with the earlier hybrid architecture could replace password-based key wrapping with a public-key/ML-KEM-based key-establishment step for authorized recipients. That is a separate integration choice and is not necessary to demonstrate the symmetric CBC task.

6. Mini-AES encryption and decryption in CBC mode

This worked example uses Phan's Mini-AES variant as documented by SageMath. It is an educational 16-bit block cipher with a 16-bit key, not standard AES. The assignment gives a 32-bit plaintext, so it is split into two 16-bit blocks. Because the instructions do not define a Mini-AES variant, the convention is stated explicitly so the numerical results can be reproduced.

6.1 Given data and CBC rules

Plaintext P = 10101000001010111011111010010011

P1 = 1010100000101011 (hex A82B)

P2 = 1011111010010011 (hex BE93)

Key K = 1100110011000111 (hex CCC7)

Initialization vector IV = 0000001100111101 (hex 033D)

CBC encryption is C1 = E_K(P1 XOR IV) and C2 = E_K(P2 XOR C1). CBC decryption is P1 = D_K(C1) XOR IV and P2 = D_K(C2) XOR C1. XOR is applied bit by bit.

6.2 Round keys

For the documented Phan/SageMath Mini-AES variant, the 16-bit key is represented by four 4-bit nibbles. The key schedule produces three round keys:

K0 = 1100110011000111 (hex CCC7)

K1 = 0101010110010010 (hex 5592)

K2 = 1010011000110100 (hex A634)

The encryption sequence is AddRoundKey with K0; SubNibble; ShiftRow; MixColumn; AddRoundKey with K1; SubNibble; ShiftRow; and the final AddRoundKey with K2. The final K2 XOR is included explicitly below to avoid ambiguity.

6.3 Encrypt the first block

1. CBC input: P1 XOR IV = 1010100000101011 XOR 0000001100111101 = 1010101100010110 (AB16).
2. AddRoundKey K0: 1010101100010110 XOR 1100110011000111 = 0110011111010001 (67D1).
3. SubNibble: 1011100010010100 (B894).
4. ShiftRow: 1011100001001001 (B849).
5. MixColumn: 0110101010011011 (6A9B).
6. AddRoundKey K1: 0110101010011011 XOR 0101010110010010 = 0011111100001001 (3F09).
7. SubNibble: 0001011111101010 (17EA).
8. ShiftRow: 0001011110101110 (17AE).
9. Final AddRoundKey K2: 0001011110101110 XOR 1010011000110100 = 1011000110011010 (B19A).

Therefore, C1 = 1011000110011010.

6.4 Encrypt the second block

1. CBC input: P2 XOR C1 = 1011111010010011 XOR 1011000110011010 = 0000111100001001 (0F09).
2. AddRoundKey K0: 0000111100001001 XOR 1100110011000111 = 1100001111001110 (C3CE).
3. SubNibble: 0101000101010000 (5150).
4. ShiftRow: 0101000100000101 (5105).
5. MixColumn: 1111100110101101 (F9AD).
6. AddRoundKey K1: 1111100110101101 XOR 0101010110010010 = 1010110000111111 (AC3F).
7. SubNibble: 0110010100010111 (6517).
8. ShiftRow: 0110010101110001 (6571).
9. Final AddRoundKey K2: 0110010101110001 XOR 1010011000110100 = 1100001101000101 (C345).

Therefore, C2 = 1100001101000101.

Final ciphertext (C1 || C2): 1011000110011010 1100001101000101

6.5 Decrypt and recover the original plaintext

The inverse Mini-AES operation reverses the encryption rounds. In this variant, decryption starts by XORing the ciphertext block with K2, then applies inverse ShiftRow and inverse SubNibble, XORs K1, applies MixColumn and ShiftRow, applies inverse SubNibble, and finally XORs K0. Each intermediate value below is 16 bits.

First block

1. C1 XOR K2 = 1011000110011010 XOR 1010011000110100 = 0001011110101110 (17AE).
2. Inverse ShiftRow: 0001011111101010 (17EA).
3. Inverse SubNibble: 0011111100001001 (3F09).
4. XOR K1: 3F09 XOR 5592 = 6A9B (0110101010011011).
5. MixColumn: 1011100001001001 (B849).
6. Inverse ShiftRow: 1011100010010100 (B894).
7. Inverse SubNibble: 0110011111010001 (67D1).
8. XOR K0: 67D1 XOR CCC7 = 1010101100010110 (AB16).
9. CBC XOR with IV: AB16 XOR 033D = 1010100000101011 = P1.

Second block

1. C2 XOR K2 = 1100001101000101 XOR 1010011000110100 = 0110010101110001 (6571).
2. Inverse ShiftRow: 0110010100010111 (6517).
3. Inverse SubNibble: 1010110000111111 (AC3F).
4. XOR K1: AC3F XOR 5592 = 1111100110101101 (F9AD).
5. MixColumn: 0101000100000101 (5105).
6. Inverse ShiftRow: 0101000101010000 (5150).
7. Inverse SubNibble: 1100001111001110 (C3CE).
8. XOR K0: C3CE XOR CCC7 = 0000111100001001 (0F09).
9. CBC XOR with C1: 0F09 XOR B19A = 1011111010010011 = P2.

Recovered plaintext: 10101000001010111011111010010011

Verification: the recovered 32-bit plaintext is identical to the plaintext provided in the assignment. This confirms that the CBC chaining and inverse operations are consistent for the selected Phan/SageMath variant.

References

1. SageMath Development Team. “Mini-AES.” SageMath Reference Manual, version 10.8. https://doc.sagemath.org/html/en/reference/cryptography/sage/crypto/block_cipher/miniaes.html
2. The Python Cryptographic Authority. “Symmetric encryption.” cryptography documentation. https://cryptography.io/en/stable/hazmat/primitives/symmetric-encryption/
3. National Institute of Standards and Technology. “Advanced Encryption Standard (AES), FIPS 197, updated May 2023.” https://csrc.nist.gov/pubs/fips/197/final
4. Python Software Foundation. “secrets - Generate secure random numbers for managing secrets.” Python 3 documentation. https://docs.python.org/3/library/secrets.html
5. Pallets Projects. “Flask Documentation.” https://flask.palletsprojects.com/

Variant note: the numerical Mini-AES example adopts Phan’s Mini-AES as documented by SageMath. If the instructor uses another classroom variant, the numeric round keys and ciphertext may differ.